

1. Apelul polimorfic al metodelor

ex:

```
Shape2D s;  
Clasele Circle si Rectangle mostenesc Shape2D.
```

```
If(cond)
```

```
    S=new Circle(...);
```

```
Else
```

```
    S=new Rectangle(...);
```

```
s.draw();//polymorphic behaviour
```

Metoda draw depinde de clasa ce referentiaza obiectul s la runtime. Nu depinde de tipul declarat al lui s.

Legatura:

```
Fie var.m();
```

Step1: crtClass=clasa obiectului referentiat

Step2: daca m() este implementat de clasa crtClass atunci este apelat, altfel CrtClass=superclasa lui CrtClass si se repeta pasul 2.

Apelul polimorfic se poate produce si la colectiile heterogene astfel ca la fiecare element poate fi apelata aceeaasi metoda dar cu implementare diferita

Ex:

```
Shape2D[] shape=new Shape2D[3];
```

```
Shape[0]=new Circle(...)
```

```
Shape[1]=new Rectangle(...)
```

```
Shape[2]=new Zetia(...)
```

```
For(int i=0;i<3;i++)
```

```
    Shape[i].draw();//metoda draw poate avea implementari diferite in cele 3 clase(mai putin Zetia ca acolo resursele sunt pe disable )
```

2. Best case scenario

Describe cum se comporta un algoritm in conditii optime. Specifica cele mai bune evenimente si conditii astfel incat algoritmul are cea mai rapida executie in timp.

Ex: Un client cumpara o vrea sa cumpere o bautura de la un automat. Selecteaza bautura, introduce pretul, aparatul verifica suma si returneaza bautura dorita. In best case, clientul introduce exact suma necesara, in automat exista acea bautura etc.

3. Clase abstracte

O clasa abstracta are cel putin o metoda abstracta(nu are corp). Metodele abstracte trebuie sa fie definite in subclase. Clasele abstracte nu pot fi instantiate in obiecte. Referinte spre clasele abstracte pot fi transmise in parametrii unor metode. Utile in procesul de generalizare.

-continutul claselor abstracte:

- instante si variabile de clase (statice)

- metode abstracte si implementate

- constructori

```
public class C {
```

```
...
```

```
public void aMethod(..., Shape2D f, ...) { ... }
```

```
}
```

La apelul metodei in locul lui f trebuie sa fie o referinta spre o subclasa ce nu este abstracta, aceasta metoda prezinta un comportament polimorfic.

-----site:

O clasa abstracta este declarata cu cuvantul cheie abstract.

Daca o subclasa nu implementeaza metoda abstracta, la randul ei trebuie sa fie declarata abstracta.

Daca o clasa implementeaza o interfata dar alege sa nu-i implementeze toate metodele, ea trebuie declarata ca si abstracta.

Membrii statici pot si folositi si accesati astfel: `AbstractClass.staticMember` sau `AbstractClass.staticMethod()`

4. Copiere si clonare

Pentru ca un obiect sa poata fi clonat este necesara implementarea interfetei `Cloneable` de catre clasa ce instantiaza acel obiect. Prin clonare se creeaza o copie a obiectului camp cu camp dar fara invocarea unui constructor.

Deoarece obiectul clonat trebuie sa fie independent de sursa sa, metoda `clone()` copiaza toti membrii si ii initializeaza cu valori doar pe cei primitivi, membrii ce contin referinte spre alte obiecte ce nu pot fi copiate la randul lor nu vor fi initializati pentru a se asigura independenta.

Copierea presupune si initializarea membrilor ce refera alte obiecte.

Implicit metoda `clone()` returneaza `Object` deci este nevoie de cast explicit.

5. Colectii Java

-sunt obiecte ce pot contine alte obiecte.

Se clasifica dupa: ordonate/neordonate; concurente/neconcurente; sunt permise duplicate sau nu.

Java Collection Framework contine interfete, implementari si algoritmi specifici colectiilor.

Prin JCF se reduce efortul programatorului, se sporeste viteza si calitate programului, programe ce corespund standardului colectiilor pot si refolosite foarte usor.

Tipuri de colectii: Lists, Maps.

Listele sunt ordonate, indexate si permit duplicate.

Maps pot fi mapeaza valori la chei, pot fi ordonate sau nu, sunt sortate.

Principalele interfete din JCF:

`Iterable`, `Collection`, `Set`, `List`, `Queue`, `Map`.

Majoritatea interfetelor declara 3 tipuri de operatii: inserarea si stergerea elementelor prin `index`, returnarea unui element de la o pozitie, iterarea prin elementele colectiei respective.

Structuri de date implementate:

Arrays, `LinkedList`, `HashTable`, `Tree`.

Colectiile ofera 2 tipuri de constructori: implicit prin care este creata o colectie goala sau cu parametru prin care dupa crearea colectiei sunt introduse valorile specificate de parametru. De obicei parametrul este o alta colectie, in speta `Set`.

Iteratorii sunt colectii speciale menite pentru traversarea obiectelor de tip colectie la cererea unor alte obiecte.

Seturile sunt colectii ce nu pot avea duplicate. Metodele `add` si `addAll` au implementari specifice.

Listele sunt ordonate, pot contine duplicate; operatii specifice: acces pozitional, cautare, iterare, `range-view`.

Maps mapeaza chei la valori, sunt o colectie de `Entries`.

6. Concepte si paradigme OO

-Conceptele de obiecte, clase, transmitere de mesaje si mostenire sunt cunoscute ca făcând parte din

paradigma orientării pe obiecte.

-Ascunderea informatiei, proiectarea dirijata de responsabilitati, impact asupra intretinerii si dezvoltarii.

Conceptele modelului calculational OO: obiect, clasa, relatiile dintre clase si obiecte.

Paradigmele modelului OO : abstractia datelor si comunicarea prin mesaje, incapsularea, polimorfismul, mostenirea.

7. Clasa Object

Este clasa mama a tuturor claselor java. Ofera metoda:

- Equals: compara doua obiecte
- hashCode: ofera codul unui al unui obiect
- notify: trezeste un thread ce asteapta dupa acest obiect
- wait: threadul curent asteapta pana cand la invocarea metodei notify
- toString: reprezentare sub forma text a obiectului
- clone: creaza o clona a obiectului

8. Clasa Class

- implementeaza interfata Serializable
- instantele acestei clase reprezinta clase si interfete java din timpul executiei unui program.
- nu are constructor public, obiectele sunt construite automat de JVM

```
void printClassName(Object obj) {
    System.out.println("The class of " + obj +
        " is " + obj.getClass().getName());
}
```

```
System.out.println("The name of class Foo is: "+Foo.class.getName());
```

9. Constructia obiectelor java

Un obiect este creat dintr-o clasă, atunci când utilizați cuvântul cheie **new**. Pentru a inițializa clasa, Java alocă memorie pentru obiect și stabilește o referință la obiect, astfel Java Runtime o poate urmări. Apoi, Java solicită constructorul clasei, care este ca o metoda, dar este apelata doar o singură dată, atunci când obiectul este creat. Constructorul este responsabil pentru a face orice prelucrare necesare pentru a inițializa obiect, cum ar fi initializarea variabilelor, deschiderea fișierelor sau a bazelor de date, și așa mai departe.

Crearea obiectelor se realizeaza in 3 pasi: **declarare, instatiere si initializare**.

Declararea unei variabile se face prin specificarea tipului si a numelui: **type name**;

Pentru a crea o instanță a unui obiect, trebuie utilizat unul dintre mecanismele de instanțiere Java. Unul dintre acestea este operatorul **new**. Operatorul **new** returnează o nouă instanță a unui obiect care a fost inițializat în funcție de comportamentul definit de către constructorul obiectului specificat, ca în următoarele exemple:

```
Foo anObject = new Foo();
Foo anotherObject = new Foo("This is a string argument");
```

Java furnizeaza mecanisme suplimentare pentru a crea instanțe ale obiectelor. Unul dintre ele este metoda **newInstance** a clasei `java.lang.Class`. Această metodă este folosită în felul următor:

```
Object anObject = Class.forName("com.acme.utils.Foobar").newInstance();
```

Numele complet al clasei este transmis la metoda **forName**. Exista de asemenea posibilitatea de a transmite metodei **forName**, un argument `String` care conține numele clasei. Acest lucru este foarte util atunci când numele clasei este descoperit la runtime.

Un alt mecanism de instanțiere este metoda **instantiate** din `java.beans.Beans`. Metoda **instantiate** are nevoie de doi parametri, un `ClassLoader` și numele complet al unei clase. Următorul este un exemplu al metodei **instantiate** care utilizează `ClassLoader` a clasei actuale:

```
Obiectul anObject = Beans.instantiate (this.getClassLoader (), "com.acme.utils.Foobean");
```

Ambele metode, atât **newInstance** cât și **instantiate** instruiesc Java Virtual Machine (JVM) să creeze o instanță a unei clase și să apeleze constructorul implicit. Constructorul implicit nu are nici un argument și este uneori menționat ca "non-args" constructor. Metodele **newInstance** și

instantiate presupun că un constructor implicit este definit pentru clasa care este instantiata.

10. Comparatie intre compozitie si mostenire

Agregarea evidentiaza clar toate elementele implicate, cod mai lung, operatii clare, separa mai bine conceptele.

Mostenirea foloseste mai putin cod, operatiile sunt mai dificil de inteles, unele metode sunt deja implementate si pot fi refolosite, folosirea inapropiata a metodelor superclasei, timp mai bun in executie deoarece legatura intre parinte si fiu se face la compilare pe cand la agregare legatura dintre obiecte se face la runtime.

Compozitia are loc atunci cand un obiect dintr-o clasa foloseste un obiect dintr-o alta clasa.

Mostenirea are loc atunci cand o clasa extinde o alta clasa

```
class MyClass {
```

```
    String s;
```

```
    ...
```

```
}
```

MyClass are controlul complet asupra metodelor sale. Mostenirea are loc atunci cand o clasa extinde o alta clasa.

```
class MyClass extends Superclass {...}
```

MyClass primeste toate variabilele statice, variabilele instantia, metodele statice si metodele de instantia ale Superclass, fie ca doreste fie ca nu. Constructorii nu se mostenesc. Mostenirea trebuie folosita doar atunci cand putem spune cinstit ca un obiect MyClass este un obiect Superclass.

11.comparatie Java/C++

Diferente

–C++

Fisierul header si fisierul de implementare sunt doua parti ale aceleiasi clase

Fisierul header contine interfata si o implementare partial

Atribute

Metode de tip “in-line”

–Java

Clasa interfata si clasa de implementare sunt doua entitati diferite

Interfata – fara implementare

Toata implementarea este inclusa in clasa

12.constructori in ierarhii de clase

-o ierarhie de clase este formata din superclase si subclase:

```
public class Shape2D { ... }
```

```
public class Circle extends Shape2D { ... }
```

```
public class Rectangle extends Shape2D { ... }
```

-O ierarhie de clase poate fi extinsa cu alte clase:

```
public class FilledCircle extends Circle { ... }
```

```
public class FilledRectangle extends Rectangle { ... }
```

-public FilledCircle (Point pc, double radius,int color) {

```
    super(pc, radius);
```

```
    this.color = color;
```

```
}
```

-Apelul constructorului din superclasa se face folosind cuvantul cheie “super”:

```
    super(pc, radius);
```

-este prima comanda in constructorul subclasei

-se foloseste aceeasi abordare pt constructorul clasei Circle

```
public Circle(Point pc, double radius) {
```

```
    super(pc);
```

```
        this.radius = radius;
    }
```

-limitarea nr de nivele de ierarhie la 3 sau 4

13.Collection Framework (interfete si implementari)

Colecțiile sunt clase specializate pentru memorarea și manipularea grupurilor de obiecte. Limbajul java oferă în cadrul pachetului standard java.util un set de clase și interfețe denumit Java Collection Framework specializat pentru lucrul cu colecții.

În Java colecțiile sunt tratate într-o manieră unitară, fiind organizate într-o arhitectură ce cuprinde:

- Interfețe: tipuri abstracte de date ce descriu comportamentul colecțiilor. Prin intermediul interfețelor colecțiile pot fi manipulate fără a ține cont de detaliile de implementare.
- Implementări: reprezintă implementări concrete ale interfețelor ce descriu colecțiile.
- Algoritmi: metode care efectuează diverse operații utile cum ar fi căutarea, sortarea definite pentru obiecte ce implementează interfețe ce descriu colecții. Acești algoritmi se numesc și polimorfici deoarece aceeași metodă poate fi folosită pe implementări diferite ale unei colecții.

14.Compozitia claselor

Compozitia este un caz particular de agregare. Exprima o agregare prin continere fizică. De exemplu, în agregarea mașinii - motor, motorul unei mașini poate fi înlocuit cu altul. Motorul unei mașini poate fi pus în altă mașină! Dacă însă reprezentăm relația mașină – motor ca relație de compoziție -> fiecare mașină are un motor unic, care nu poate fi înlocuit și motorul nu poate fi pus în altă mașină!

Relația de compunere este semantic echivalentă cu considerarea componentelor ca atribute ale clasei agregate.

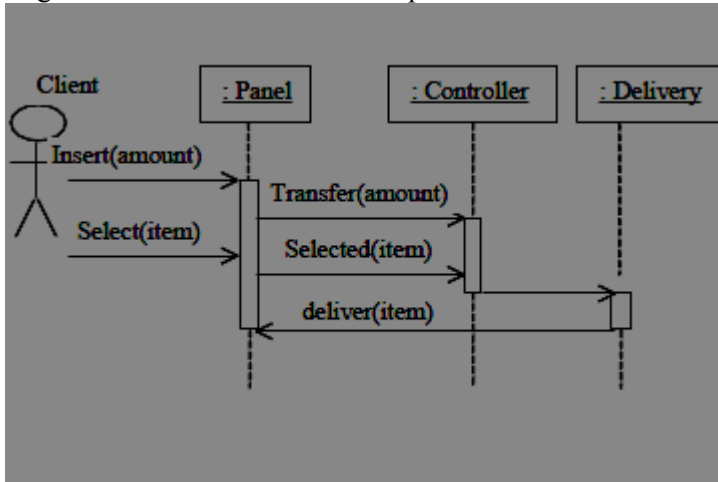
15.Diagrame de tranziție UML

-ca la diagrame de activități

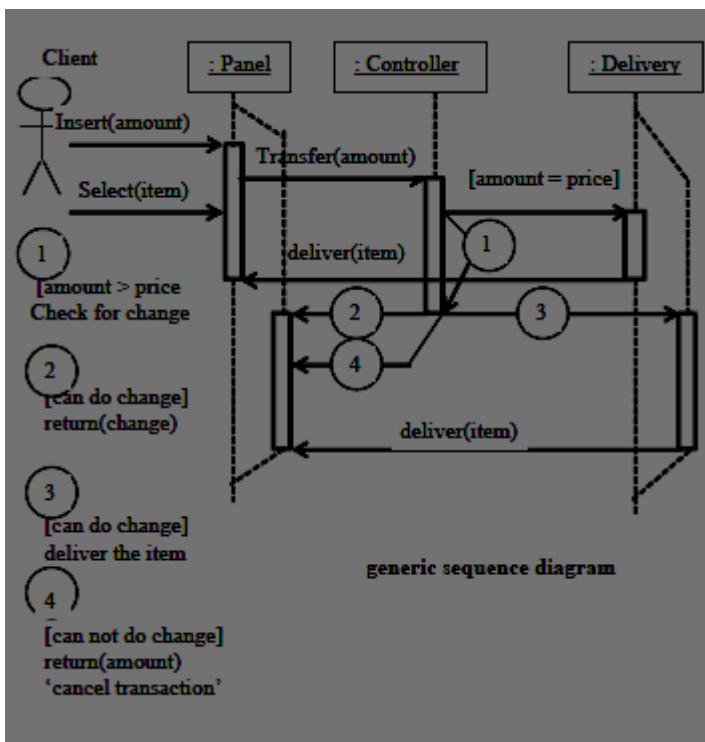
16.Diagrame de secvență UML

- ilustrează comportamentul sistemului
- Concepte:
 - Arată obiectele implicate în interacțiune
 - Lifeline (linia vieții???)
 - linie verticală punctată sub obiecte
 - arată perioada de timp în care obiectele joacă un anumit rol într-un caz de utilizare
 - Mesaj
 - săgeți indicând dinspre lifeline-ul unui obiect trimis spre lifeline-ul unui obiect primit
 - mesajele au nume și pot avea argument
 - fiecare mesaj mută fluxul de control de la un obiect la altul
 - Scenariu
 - un caz de utilizare poate conține mai multe scenarii
 - unul din scenarii este "best case"
 - celelalte scenarii pot descrie diferite evenimente și condiții în cazul de utilizare
 - Activarea
 - dreptunghi îngust sub obiect de-a lungul lifeline-ului
 - perioada de timp în care obiectul procesează mesajul

- cand un obiect termina procesarea unui mesaj, controlul se intoarce la expeditorul mesajului. Acest fapt marcheaza terminarea activarii coresp. acelui mesaj
- este marcata printr-o sageata punctata pornind din partea de jos a dreptunghiului de activare inapoi spre lifeline-ul obiectului care a trimis mesajul
- activarile si mesajele de returnare sunt optionale in diagrama de secventa
- In cursul procesarii mesajelor un obiect poate trimite mesaje spre alte obiecte
- diagrama instanta de secventa: acopera doar scenariul “best-case”



- diagrama generica de secventa:
 - ia in considerare fluxul de control
 - permite mai multe scenarii in diagrama
 - fiecare conditie cauzeaza un ‘fork’ de control in mesaj, separand mesajul in cai
 - conditiile sunt reprezentate intre paranteze drepte: [amount=price]

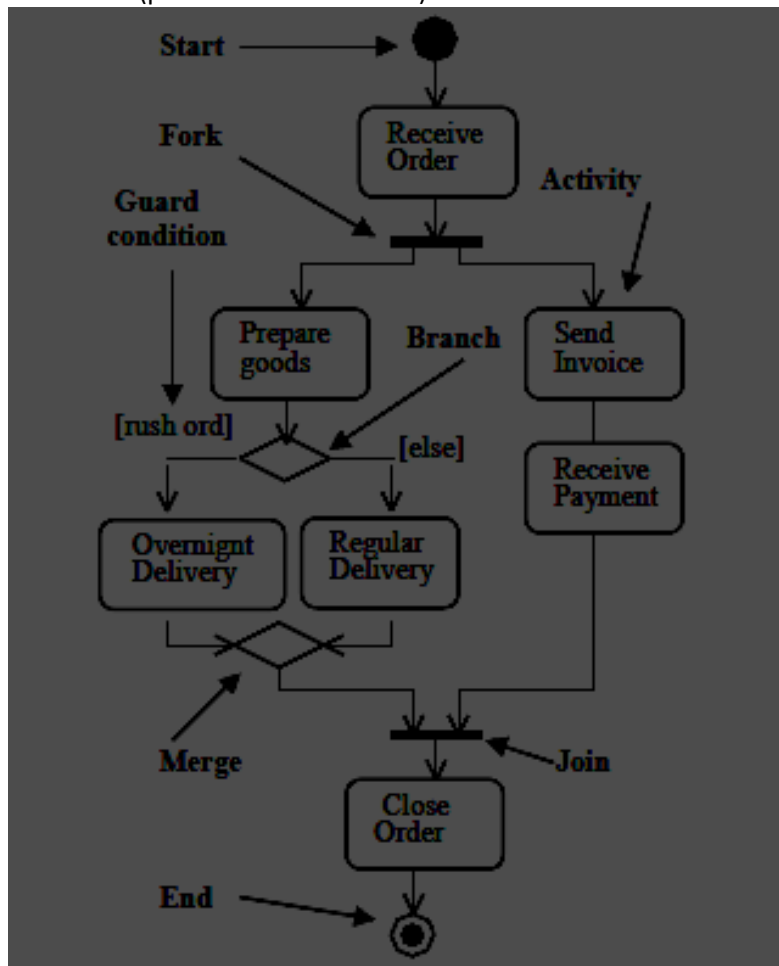


17. Diagrama de activitate UML.

=ilustreaza fluxul evenimentelor intr-un caz de utilizare

- Concepte
- activitatea – o stare de indeplinire a unor sarcini (o metoda a unei clase, o activitate din lumea reala)
 - diagrama de activitate descrie secvente de activitati
 - suport pentru comportament conditional si paralel
- tranzitia

- ramificatia(branch)
 - un punct de decizie in care sunt 2 sau mai multe cai posibile ale fluxului de control
 - tranzitie unica de intrare
 - mai multe tranzitii de iesire pazite (guarded)
 - ramificatia este reciproc exclusiva
 - [else] –toate celelalte sunt false
 - guard – expresie booleana
- contopire (merge)
 - mai multe tranzitii de intrare
 - iesire unica
 - marcheaza sfarsitul unui comportament conditional
- bifurcare(fork)
 - o tranzitie de intrare unica
 - mai multe tranzitii de iesire
 - când tranziția de intrare este declanșata, tranzițiile de ieșire sunt luate în paralel
 - poate specifica programare concurenta
- reuniune(join)
 - mai multe tranzitii de intrare
 - iesire unica
 - rol de sincronizare
- simbolurile de reprezentare a principalelor concepte sunt prezentate în diagrama de activitate urmatoare (procesarea comenzilor):

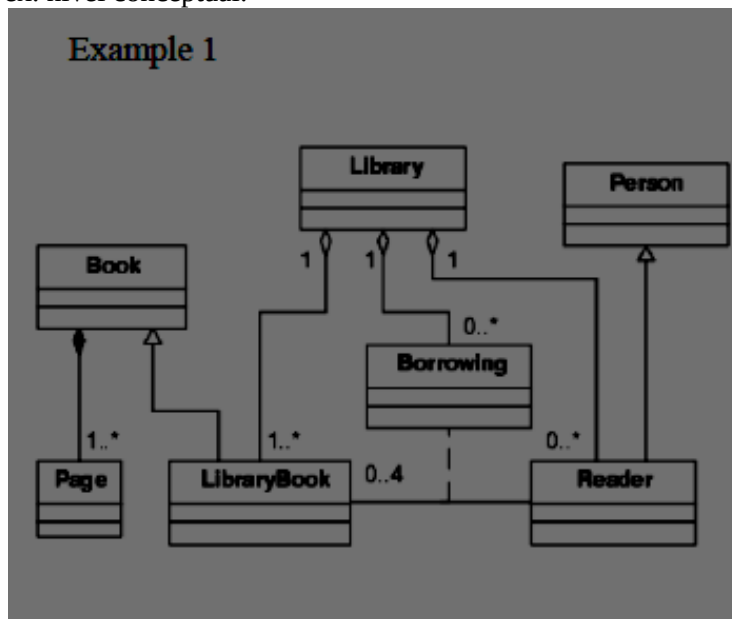


18.Diagrame de clase.

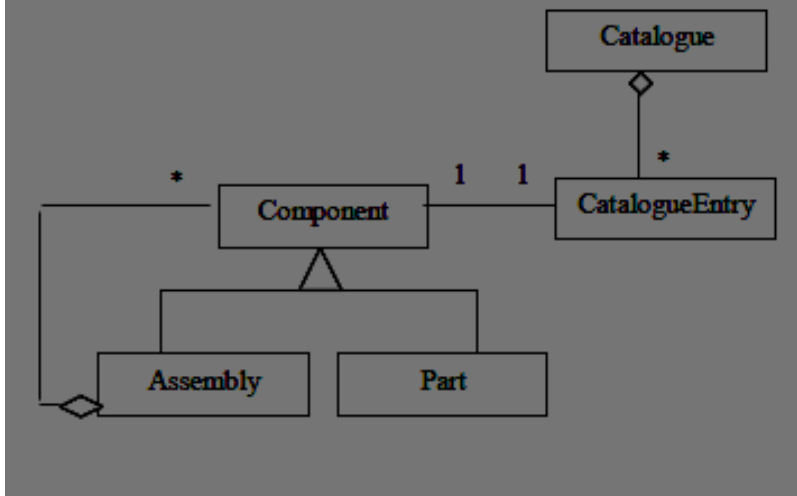
- ilustreaza sructura logica a sistemului
- descriu clasele si obiectele ce apartin sistemului, precum si relatiile dintre acestea
 - relatii statice

- asocieri
 - agregare
 - generalizare (mostenire)
- pentru o clasa poate ilustra:
 - attribute
 - metode
 - constrangeri
- 3 perspective:
 - Conceptuala
 - deseneaza o diagrama reprezentand conceptele din domeniul problemei
 - nu este relationata cu implementarea
 - se axeaza pe:
 - functionalitate
 - relatiile dintre concepte
 - Specificatie
 - perspectiva software
 - abordare comportamentala (interfete), nu implementare
 - Implementare
 - clase propriu-zise avand attribute si metode
 - Note:
 - limitele dintre cele 3 abordari nu sunt foarte clare
 - nu sunt mari diferente intre “conceptuala” si “specificatie”
 - sunt diferente notabile intre diagramele de specificatie si cele de implementare

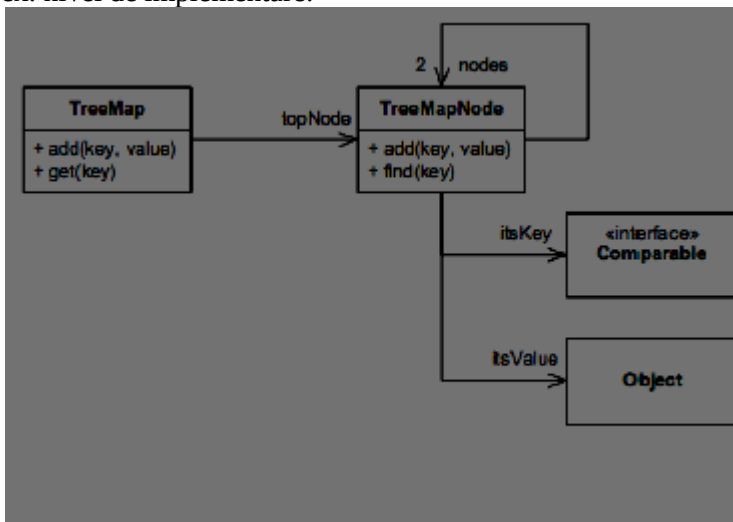
ex. nivel conceptual:



Example 2



ex. nivel de implementare:



19.Exceptiile si folosirea lor in Java.

O exceptie este un eveniment ce se produce în timpul executiei unui program si care provoaca întreruperea cursului normal al executiei.

Exceptiile pot aparea din diverse cauze si pot avea nivele diferite de gravitate: de la erori fatale cauzate de echipamentul hardware pâna la erori ce tin strict de codul programului, cum ar fi accesarea unui element din afara spatiului alocat unui vector. In momentul când o asemenea eroare se produce în timpul executiei sistemul genereaza automat un obiect de tip exceptie ce contine:

- Informatii despre exceptia respectiva
- starea programului în momentul producerii acelei exceptii

Crearea unui obiect de tip exceptie se numeste aruncarea unei exceptii ("throwing an exception").

In momentul în care o metoda genereaza o exceptie (arunca o exceptie) sistemul de executie este responsabil cu gasirea unei secvente de cod dintr-o metoda care sa trateze acea exceptie. Cautarea se face recursiv, începând cu metoda care a generat exceptia si mergând înapoi pe linia apelurilor catre acea metoda.

Secventa de cod dintr-o metoda care trateaza o anumita exceptie se numeste analizor de exceptie ("exception handler") iar interceptarea si tratarea exceptiei se numeste prinderea exceptiei ("catch the exception").

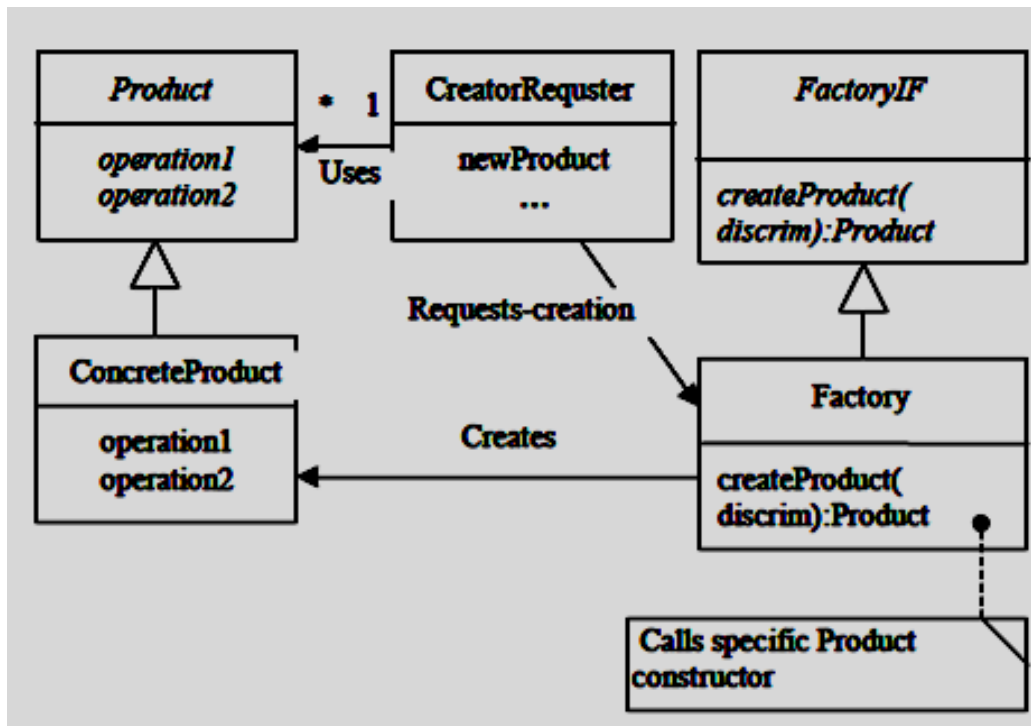
Cu alte cuvinte la aparitia unei erori este "aruncata" o exceptie iar cineva trebuie sa o "prinda" pentru a o trata. Daca sistemul nu gaseste nici un analizor pentru o anumita exceptie atunci programul Java se opreste cu un mesaj de eroare.

20.Exceptii: utilizare si tratare

- O exceptie este o problema care apare in cursul executiei unui program. La aparitia unei exceptii JVM creeaza un obiect de clasa 'Exception' care contine informatii despre problema aparuta. Insi programul Java poate intercepta (catch) o exceptie. Apoi poate folosi obiectul de tipul exceptie pentru a-si reveni dupa problema.
 - Obs: Exceptii != Erori
 - La aparitia unei exceptii JVM creeaza un obiect de clasa Exception care contine informatii despre problema aparuta.
 - Insi programul Java poate intercepta (catch) o exceptie. Apoi poate folosi obiectul de tipul exceptie pentru a-si reveni dupa problema.
 - Exemplu: `int inputNumber = keyboard.nextInt();` pt intrare: h1
 - Calea fundamental pentru tratarea exceptiilor în Java constă din trio-ul try-throw-catch
 - Blocul try contine codul pentru algoritmul implementat
 - Acest cod spune ce se face atunci când totul merge bine
 - Se numeste bloc try deoarece el "încearcă" să execute cazul în care totul merge asa cum a fost planificat
 - De asemenea acest bloc poate contine cod care aruncă o exceptie dacă se întâmplă ceva neobisnuit
- ```
try {
 CodCarePoateAruncaOExceptie
}
catch(Exception e) { . . . }
```
- Identificatorul e din blocul catch de deasupra se numeste parametru al blocului catch
  - Parametrul blocului catch îndeplineste două roluri:
    1. Specifică tipul de obiect exceptie aruncat pe care blocul catch îl poate intercepta (d.e., mai sus este un obiect de clasa Exception)
    2. Oferă un nume (pentru obiectul care este interceptat) care să fie folosit în blocul catch
  - Observatie: adesea se foloseste identificatorul e prin conventie, dar se poate folosi orice identificator care nu este cuvânt cheie.

## 21.Factory Method

- este un pattern creational
- se ocupa cu problema crearii obiectelor fara a specifica exact clasa obiectului care va fi creat
- defineste o interfata pentru crearea obiectelor dar lasa subclasele sa decida care clasa sa fie instantiata.
- permite unei clase sa diferentieze procesul de instantiere al subclaselor
- clasa C instantiaza alte clase fara sa fie dependenta de vreo clasa pe care o instantiaza
- clasa C ramane independenta de clasele instatiate, deleaga alegerea care clasa sa fie instantiata altui obiect, noul obiect creat este referit printr-o interfata comuna
- FactoryIF este o interfata/clasa abstracta care e subclasata de alte clase concrete (Factory)
- exista o singura metoda creatProduct concreta pentru un singur tip de produs.



## - Participanti

### Product

- superclasa abstracta ale obiectelor produse de patternul Factory Method
- este numita de obicei: Document, Image, etc

### Concrete Product

- orice clasa concreta instantiata de obiecte care participa in patternul Factory Method
- este numita de obicei RTFDocumentor JPEGImage

### FactoryIF

- clasa abstracta independenta a aplicatiei
- declara o metoda care este apelata de un obiect CreateRequester pentru crearea obiectelor concrete (de obicei este numita createDocument / createImage)
- argumentele metodei sunt folosite pentru a deduce care clasa va fi instantiata
- numita de obicei: DocumentFactoryIF, ImageFactoryIF

### CreationRequester

- clasa independenta a aplicatiei
- trebuie sa creeze obiecte specifice aplicatiei. Face asta direct printr-o instanta a clasei Factory

### Factory

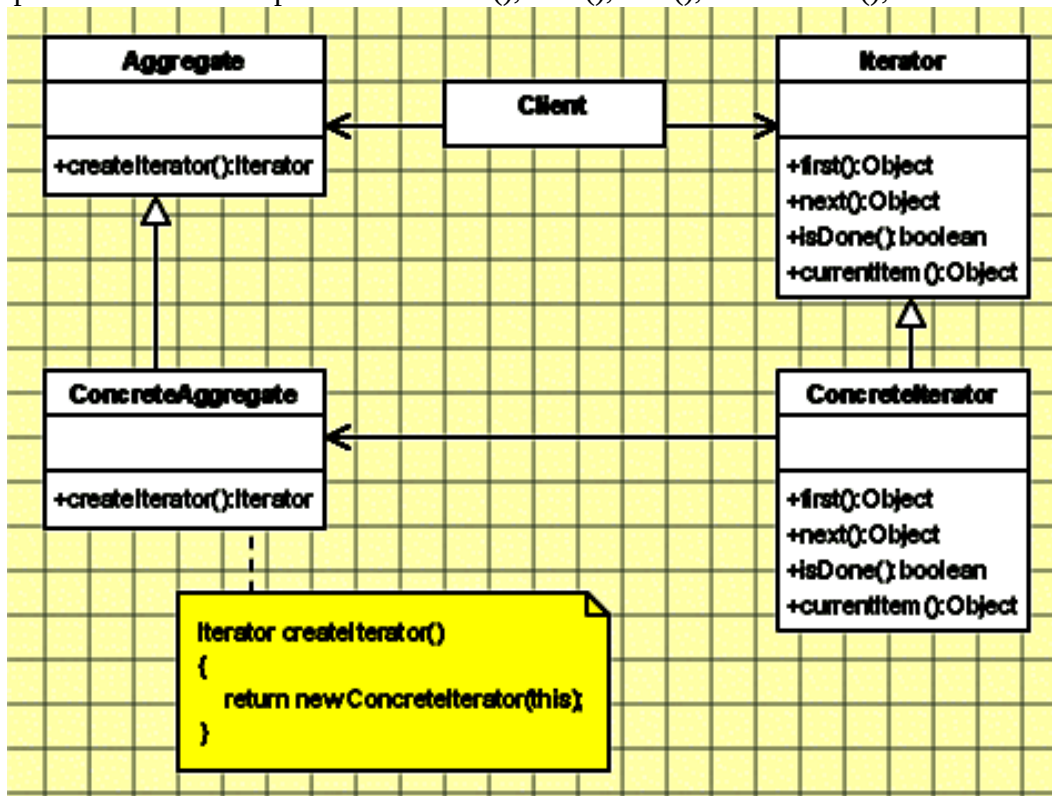
- clasa specifica a aplicatiei
- subclasa a clasei abstracte si are o metode pentru crearea obiectelor ConcreteProduct
- numita de obicei DocumentFactory, ImageFactory

## 22.Iterator - design pattern

- pattern comportamental
- incapsuleaza felul in care se traverseaza si acceseaza componentele unui obiect agregat (colectie)
- o colectie este o grupare de obiecte de acelasi tip sau obiecte exprimate prin acelasi tip de baza (de exemplu Object). Iterator ofera o modalitate de accesa aceste elemente fara a expune structura lor interna. Are un mecanism de traversare asemanator cu cel al traverarii listelor sau array-urilor.
- permite:

- Accesarea continutului unei colectii fara a expune structura sa interna
- Suporta multiple traversari simultane a colectiei

- Ofera o interfata uniforma pentru traversarea diferitelor colectii
- poate folosi metode precum: hasNext(), next(), first(), currentItem();



## 23. Metoda equals

- Contracts:

- **Reflexivitatea** : pentru orice valoare a lui x diferita de null x.equals(x) returneaza true
- **Simetria**: pentru orice valori ale lui x si y diferite de null, x.equals(y) returneaza true daca si numai daca y.equals(x) returneaza true.
- **Tranzitivitatea**: pentru orice valori ale lui x, y, z diferite de null, daca x.equals(y) returneaza true si y.equals(z) returneaza true atunci si x.equals(z) va returna true.
- **Consistenta**: pentru orice valori ale lui x, y diferite de null, multiplele invocari ale lui x.equals(y) returneaza in mod constant true sau returneaza in mod constant false
- **Non-nulitate**: pentru orice valoare a lui x diferita de null, x.equals(null) returneaza false

-skeleton:

```

public class C {
 // ... class resources
 public boolean equals(Object o)
 {
 if (o == this) return true;
 if (!(o instanceof C)) return false;
 C cObj = (C) o;
 return ... ; // logical test of equality
 }
}

```

-metoda equals este definita in clasa Object

-la suprascrierea metodei equals este necesara si suprascriere metodei hashCode() astfel incat 2 obiecte egale trebuia sa aiba hashCode-uri egale.

-problemele apar in cazul extinderii unei clase instantiabile si adaugarea unui atribut in plus. De exemplu avem clasa Point si clasa ColorPoint care extinde clasa Point dupa cum urmeaza:

**public class Point**

```
{
private final int x;
private final int y;
public Point(int x, int y)
{ this.x = x; this.y = y; }

public boolean equals(Object o)
{
 if (!(o instanceof Point) return false;
 Point p = (Point)o;
 return p.x == x && p.y == y;
}
}
```

**public class ColorPoint extends Point**

```
{
private Color color;
public ColorPoint(int x, int y, Color color)
{
 super(x, y);
 this.color = color;
}
}
```

-se pune problema asupra modului in care va fi implementata metoda equal in clasa ColorPoint

```
public boolean equals(Object o)
{
 if (!(o instanceof ColorPoint) return false;
 ColorPoint cp = (ColorPoint)o;
 return super.equals(o) && cp.color == color;
}
```

-in acest caz pentru 2 obiecte:

```
Point p = new Point(1, 2);
ColorPoint cp = new ColorPoint(1, 2, Color.RED);
```

p.equals(cp) => true, iar cp.equals(p) => false, ceea ce violeaza simetria metodei equals

-o alta varianta de rescriere este:

```
public boolean equals(Object o)
{
 if (!(o instanceof Point)) return false;
 // If o is a normal Point, do a color-less comparison
 if (!(o instanceof ColorPoint)) return o.equals(this);
 ColorPoint cp = (ColorPoint)o;
 return super.equals(o) && cp.color == color;
}
```

-in acest caz pentru 3 obiecte:

```
ColorPoint p1 = new ColorPoint(1, 2, Color.RED);
```

```
Point p2 = new Point(1, 2);
```

```
ColorPoint p3 = new ColorPoint(1, 2, Color.BLUE);
```

p1.equals(p2) => true, p2.equals(p3) => true, insa p1.equals(p3) => false ceea ce violeaza tranzitivitatea metodei equals.

-nu exista solutie pentru aceasta problema insa exista metode pentru evitarea aparitiei problemei, de exemplu in acest caz clasa ColorPoint nu va extinde clasa Point ci va contine o variabila de instanta de tipul Point dupa cum urmeaza:

```
public class ColorPoint
{
 private Point point;
 private Color color;
 public ColorPoint(int x, int y, Color color)
 {
 point = new Point(x, y);
 this.color = color;
 }
}
```

## 24.Ce este o exceptie RunTime? Dati exemple

-exceptii care pot fi aruncate in timpul functionarii normale a Java Virtual Machine

-sunt de tipul java.lang.RunTimeException

-exemple: NullPointerException, ArrayIndexOutOfBoundsException

-pot fi prevenite prin intermediul programului. De exemplu daca se verifica nulitatea inainte de fiecare apelare a unei metodei, NullPointerException nu va aparea niciodata, si similar, ArrayIndexOutOfBoundsException nu va aparea niciodata daca se verifica intai indexul.

## 25.Explicati: „Java este neutru arhitectural.”

Neutralitatea fata de arhitectura inseamna, de fapt, ca arhitectura masinii virtuale Java, care este ea insasi un calculator abstract (organizarea memoriei, conventiile de reprezentare a datelor in memorie, setul de instructiuni al procesorului etc) nu depinde de arhitectura calculatorului pe care aceasta se instaleaza. Intrucat codul de octeti obtinut din compilarea programului Java nu se executa nemijlocit pe calculatorul gazda, ci pe masina virtuala Java, el poate fi acelasi pe orice calculator pe care este instalata o astfel de masina, deci este neutru fata de arhitectura calculatorului gazda.

## 26.Generics in care sa explicam principile de puts si gets

-Wildcard:

? extends E

- Permite adaugarea membrilor altei colectii care are elemente de tipul E

- Permite adaugarea de elemente care sunt instante ale unor subtipuri ale lui E

? se numeste wildcard

## Principiile Get si Put:

-se foloseste wildcard **extends** atunci cand doar se preiau valori dintr-o structura (get)

-se foloseste wildcard **super** atunci doar se adauga valori intr-o structura (put)  
-**nu** se foloseste wildcard atunci cand se si adauga si se pe preiau valori din acceasi colectie (get and put)

```
public static <T> void copy(List<? super T> dest, List<? extends T> src)
```

-metoda preia valori din sursa src, deci este declarat cu wildcard extends  
-metoda adauga valori in destinatia dest , deci este declarat cu wildcard super

- **oricand se foloseste un iterator, se preiau valori dintr-o structura, deci se foloseste wildcard extends**

-metoda sum: primeste ca parametru o colectie de obiecte Number, le transforma in double si le aduna

```
public static double sum(Collection<? extends Number> nums) {
 double s = 0.0;
 for (Number num : nums) s += num.doubleValue();
 return s;
}
```

Daca este folosit extends, atunci urmatoarele apeluri sunt permise:

```
List<Integer> ints= Arrays.asList(1,2,3);
assert sum(ints) == 6.0;
List<Double> doubles = Arrays.asList(2.78,3.14);
assert sum(doubles) == 5.92;
List<Number> nums= Arrays.<Number>asList(1,2,2.78,3.14);
assert sum(nums) == 8.92;
```

Primele doua apeluri nu ar fi fost permise daca nu era folosit extends

- **oricand se foloseste metoda add se adauga valori intr-o structura, deci se va folosi wildcard super**

-metoda count primeste ca parametru o colectie de obiect de tip Number si un numar intreg n si adauga primele n valori intregi incepand cu zero in colectie

```
public static void count(Collection<? super Integer> ints, int n) {
 for (int i = 0; i < n; i++) ints.add(i);
}
```

Daca este folosit super, atunci urmatoarele apeluri sunt permise:

```
List<Integer> ints = new ArrayList<Integer>();
count(ints, 5);
assert ints.toString().equals("[0, 1, 2, 3, 4]");
List<Number> nums = new ArrayList<Number>();
count(nums, 5); nums.add(5.0);
assert nums.toString().equals("[0, 1, 2, 3, 4, 5.0]");
List<Object> objs = new ArrayList<Object>();
count(objs, 5); objs.add("five");
assert objs.toString().equals("[0, 1, 2, 3, 4, five]");
```

Ultimele doua apeluri nu ar fi fost permise daca nu era folosit super.

- **oricand se folosesc ambele metode put si get nu se foloseste un wildcard**

```
public static double sumCount(Collection<Number> nums, int n)
{
 count(nums, n);
 return sum(nums);
}
```

Colectia este trecuta atat prin sum cat si prin count, deci elementele sale trebuie sa satisfaca conditiile:

- amandoua sa extinda Number (necesar pentru sum)
- sa fie super pentru Integer (necesar pentru count)
- clasele care satisfac aceste conditii sunt: Number si Integer. Consideram Number.

```
List<Number> nums= new ArrayList<Number>();
double sum = sumCount(nums,5);
assert sum == 10;
```

Daca nu exista un wildcard, argumentul trebuie sa fie o colectie de obiecte Number.

- principiile Get si Put functioneaza si in sensul opus:
- daca se foloseste extends => se pot prelua valori dar nu se pot adauga
- daca se foloseste super => se pot adauga valori dar nu se pot prelua

```
List<Integer> ints= Arrays.asList(1,2,3);
List<? extends Number> nums= ints;
double dbl = sum(nums); // ok
nums.add(3.14); // compile-time error
```

- se poate apela sum (se pot prelua valori)
- nu se poate apela add (nu se pot adauga valori)

```
List<Object> objs = Arrays.<Object>asList(1,"two");
List<? super Integer> ints = objs;
ints.add(3); // ok
double dbl = sum(ints); // compile-time error
```

- se poate apela add (se pot adauga valori)
- nu se poate apela sum (nu se pot prelua valori)

### **Exceptii:**

1. Nu se poate pune nimic intr-un tip declarat cu wildcard extends cu exceptia valorii null  
Null apartine fiecarui tip referit

```
List<Integer> ints = Arrays.asList(1,2,3);
List<? extends Number> nums = ints;
nums.add(null); // ok
assert nums.toString().equals("[1,2,3,null]");
```

2. Nu se poate prelua nimic dintr-un tip declarat cu wildcard super cu exceptia unei valori de tipul Object

-Object este supertip pentru orice obiecte de referinta

```
List<Object> objs = Arrays.<Object>asList(1,"two");
List<? super Integer> ints = objs;
String str = "";
for (Object obj : ints) str += obj.toString();
assert str.equals("1two");
```

- `?` extends `T`  
-contine fiecare tip dintr-un interval delimitat de `null` in partea de jos si `T` in partea de sus
- `?` super `T`  
- contine fiecare tip dintr-un interval delimitat de `T` in partea de jos si `Object` in partea de sus

## 27.Incapsulare

- detaliile de implementare ale unui obiect nu sunt relevante altor obiecte, celelate obiecte fiind constiente doar de comportamentul obiectului respectiv.
- este o tehnica prin care campurile unei clase sunt private, iar accesul la campuri se face prin metode publice. Cand un camp al unei clase este privat, acesta nu poate fi accesat de alte clase. In mod normal, metode prin care se acceseaza acesti campuri private sunt getter-ele si setter-ele.
- incapsularea este un mod de ascundere a datelor
- poate fi descrisa ca o bariera protectoare care previne accesul la date si cod al unui obiect definit in afara clasei, astfel o clasa are control absolut asupra campurilor sale.

## 28.Interfete Java

- este un tip abstract, ce descrie un comportament al claselor care o implementeaza
- sunt declarate folosind cuvantul cheie *interface*, si poate contine numai semnaturi ale metodelor sale, care trebuie sa fie publice si declaratii de constante (variabile declarate *static final*).
- nu contine definitii pentru metode
- nu poate fi instantiata
- o clasa care nu este abstracta si implementeaza o interfata trebuie sa implementeze toate metodele descrise in interfata
- obiectele de tipul unei interfete trebuie fie sa fie `null` fie sa fie legate de un obiect care implementeaza interfata.
- o interfata nu poate sa implementeze alta interfata, dar poate sa extinda mai multe interfete
- o clasa poate sa implementeze mai multe interfete

## 29.Interfete marker

- sunt interfete goale, care nu contin nici o metoda
- folosite pentru adaugarea de functionalitati pentru clasele care le implementeaza
- exemple de interfete marker: `Cloneable` (face distinctia intre clasele care pot fi clonate si care nu), `Serializable` ;

```
public interface M {}
public class C implements M { ... }
```

M-este o interfata goala

C-este marcata ca avand o anumita proprietate, proprietatea este data de numele interfetei

## 30.Mecanismul de reutilizare software: mostenire si compozitie.

### -Compozitie:

- relatia de compozitie reprezinta o relatie de tipul intreg-parte si este o forma a agregarii.
- specifica ca timpul de viata al partii este dependent de timpul de viata al intregului.

- de obicei datele circula intr-o singura directie si anume de la intreg la parte.
- o parte a starii intregului este reprezentata prin intermediul partii.
- compozitie dinamica: legaturile dintre noile abstractii si abstractiile vechi sunt create/ schimbate la run-time
- avantaje:
  - tehnica simpla
  - toate elementele implicate sunt clar evidentiate
  - sunt foarte usor de schimbat folosind alte componente structurale
- dezavantaje
  - cod mai lung

### 31.Mostenirea.Mostenirea in Java.

- modalitate de compartimentare si reutilizare a codului prin crearea unor colectii de attribute si comportamente, numite obiecte care se poate baza pe obiecte create anterior. In Java este permis ca o clasa sa mosteneasca alte clase, clasele noi fiind cunoscute sub numele de subclase.
- subclasele mostenesc attributele si comportamentul din clasele pre-existente, numite superclase
- cu exceptia clasei Object, care nu are nici o superclasa, fiecare clasa are numai o superclasa directa (mostenire unica). In absenta unei superclase explicite, fiecare clasa are ca si superclasa implicita clasa Object
- relatiile de mostenire dau nastere unei ierarhi de clase
- o subclasa poate defini noi valori ale datelor, noi metode, sau poate rescrie metodele din superclasa.
- pentru a crea o subclasa, prgramtorul trebuie sa studieaze si sa inteleaga metodele din superclasa
- legatura dintre clasa copil si clasa parinte este stabilita in momentul compilarii
- avantaje:
  - mai putin cod.
  - unele metode sunt deja implementate si pot fi reutilizate
  - timp de executie mai bun
  - permite folosirea noii abstractii ca argument intr-o metode polimorfica existenta.
- dezavantaje:
  - predispus la erori
  - folosirea nepotrivita a metodelor unei clase
  - diferente semantice
  - operatiile sunt mai greu de inteles

### 32.Mostenirea selectiva

- folosita impreuna cu clasele adaptoare (Adapter classes)
- clasa adaptoare contine metode cu un comportament default (de obicei sunt metode goale)
- clasele care extind clasa adaptoare implementeaza doar anumite metode ale superclasei.

```
public class Adapter
{
public int m1(...) { }
public double m2(...) { }
public String m3(...) { }
// ... other class resources
}
```

```
public class Alpha extends Adapter
```

```

{
public int m1(...) {
// ... m1 specific implementation
}
public String m3(...) {
// ... m3 specific implementation
}
// ... other class resources
}

```

#### **public class Beta extends Adapter**

```

{
public int m1(...) {
// ... m1 specific implementation
}
double m2(...) {
// ... m2 specific implementation
}
// ... other class resources
}

```

### **33.Modeling semantic gap**

- semantic gap (diferenta semantica) caracterizeaza diferenta intre 2 descrieri ale unui obiect de catre diferite reprezentari lingvistice. Mai exact, înseamnă diferența dintre formularea ambiguă a unui obiect într-un limbaj natural si reprezentarea ei într-un limbaj de programare
- conceptul este relevant atunci cand activitatile umane obisnuite, sau anumite sarcini sunt transferate intr-un limbaj de programare.
- modelarea diferentelor semantice se face prin programe care reduc decalajul dintre cele doua descrieri ale unui obiect.

### **34.Overriding si overloading**

**Overriding:** intre clase si subclase. Metodele din subclase au aceleasi elemente cu metodele din superclase(nume, semnatura sau tipul returnat).

Exemplu:

Class A { public void m{...} } si Class B extends A { public void m{...}}

Metoda m din clasa A este suprascrisa de metoda m din clasa B. Pentru un obiect dat, doar un “m” se poate invoca.

**Overloading:** intre metodele aceeasi clase(aceiasi nume, semnături diferite-adica tipul si parametrii).

Exemplu:

```

public int test(int units, int pricePerUnit) {
return units*pricePerAmount;
}

```

```
public long test(int numUnits, int weight) {
return (long)(units*weight);
}
```

Suprascrierea unei metode mostenite se face prin scrierea unei alte metode aceeasi semnatura. Suprascrierea unei metode nu se poate face mai putin publica (public->protected->package->private), nu putem arunca exceptii suplimentare dar putem arunca mai putine, tipurile returnate trebuie sa fie compatibile.

Tot suprascriere este cand dintr-o clasa A care extinde o clasa B se apeleaza “super.metoda()”. Astfel o subclasa invoca o metoda din superclasa.

### **35.Observer – Design Pattern**

Defineste o dependenta “one-to-many” , 1:n, intre obiecte. Cand un obiect isi schimba starea, toate cele n obiecte care depind de el sunt instiintate/notificate si updatate automat.

Design patternul Observer trebuie folosit atunci cand un “subiect” trebuie sa fie observant de unul sau mai multi observatori. Astfel obiectul numit “subiect” detine o lista cu obiectele dependente, numite observatori si le informeaza automat la fiecare schimbare de stare, de obicei, prin apelul unui metode a acestora. Este folosit în principal pentru punerea în aplicare a sistemelor distribuite de manipulare a evenimentelor.

Cand o anumita abstractizare are doua componente dependente, cele doua component ar trebui implementate ca doua clase (separate/separat) dependente. Cand schimbarile unui obiect determina schimbari altor obiecte dependente, numarul obiectelor dependente este necunoscut.

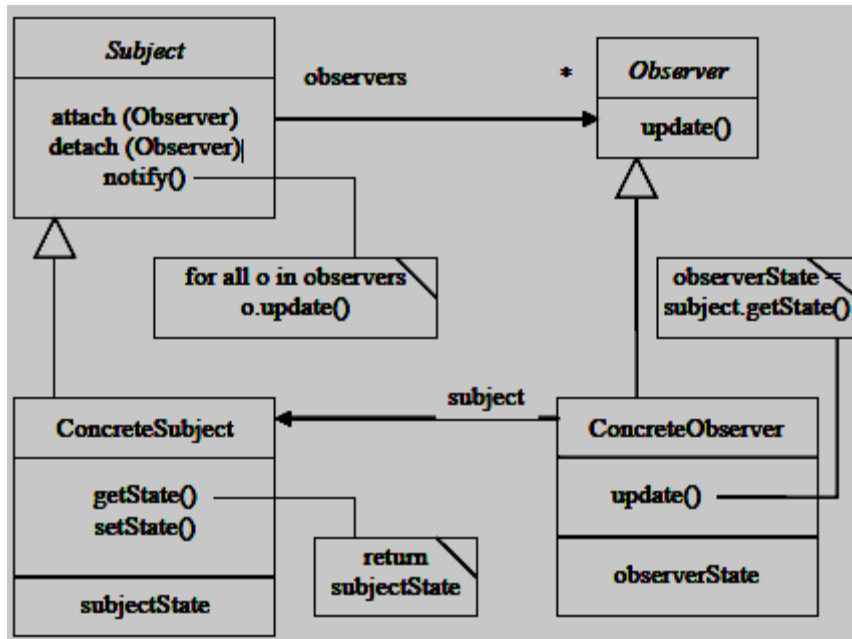
Participanti:

-Subject – mai multe obiecte Observer pot observa obiectul Subject

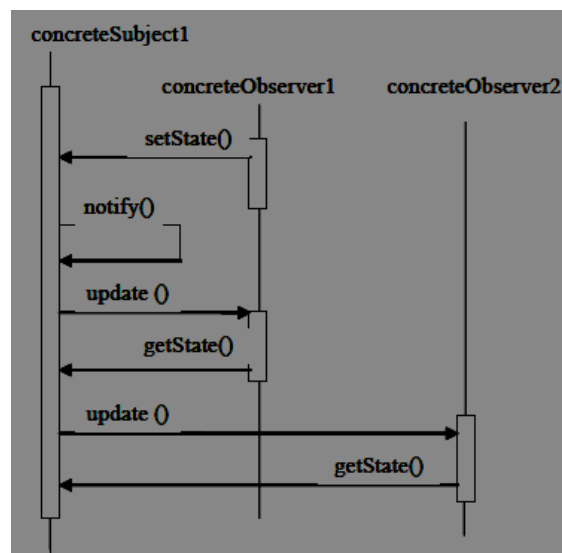
-Observer – specifica o interfata care ar trebui implementata de ConcreteObservers , care trebuie informata/notificata oricand are loc o schimbare la subiectul urmarit

-ConcreteSubject – stocheaza/contine o stare de care sunt interesati observatorii; Notifica observatorii cand starea se modifica.

-ConcreteObserver – implementeaza interfata de update (Observer); Sunt notificati cand un subiect isi schimba starea; Gestioneaza o referinta spre un ConcreteSubject, in care arata interesul; Stocheaza/contine o stare care ar trebui sa se potriveasca cu starea din ConcreteSubject .



Colaborarea Participantilor:



Consecinte:

-Datorita cuplajului slab dintre subiecti si observatori: observatorii pot fi adaugati/stersi fara a se modifica subiectii acestora; subiectii si observatorii pot face parte din diferite straturi de abstractizare

-Suport pentru evenimente de broadcast: Subiectii trimit notificari la toti observatorii subscrisi(/care au subscris); Observatorii pot fi adaugati sau stersi oricand.

### 36. Polimorfism

Polimorfism = abilitatea unui obiect de a lua mai multe forme. Polimorfismul, dupa Cardelli si Wegner poate fi Universal(Parametric sau de Incluziune) sau Ad-hoc(Constrangere sau Suprascrisiere).

Legatura statica – verificare de tip statica – garanteaza corectitudinea; interpretare rigida

-Verificare de tip dinamica – combinative invalida

Legatura dinamica – verificare de tip statica – garanteaza corectitudinea; interpretare flexibila

- Verificare de tip dinamica – nu garanteaza corectitudinea; interpretare flexibila

In programul C – partea stanga si partea dreapta a assignmentului(a atribuirii) trebuie sa fie de tipuri compatibile

In programul JAVA – toate obiectele sunt polimorfe; atribuire polimorfica; forma puternica de atribuire; exista o norma de atribuire.

Polimorfismul permite folosirea unui obiect in locul altui obiect (a unei subclase in locul superclasei). Polimorfismul este capacitatea unei actiuni sau a unei metode de a face lucruri diferite bazate pe obiectul pe care il interpreteaza. Overloading, Overriding si Dynamic method sunt 3 tipuri de polimorfism. Overriding – are loc cand o metoda a clasei are acelasi nume si aceeasi semnatura ca a unei metode din clasa mama. Overloading – are loc cand mai multe metode au acelasi nume dar semnatura difera.

Exemplu: Aparat -> Electronic, Electric

### **Legarea dinamica**

Este o proprietate a limbajului Java( la C# lipseste). Comportamentul fizic este luat in ultima instanta posibila – RunTime(adica la executie). In locul unei definitii prefer sa dau un exemplu pentru a explica ce este legarea dinamica. Consideram ca avem un program java numit sample.java , care contine trei clase, prima fiind clasa main: sample, o clasa numita add, si una numita sub . Acum incercam sa compilam programul folosind compilerul java. Observam ca au aparut 3 fisiere .class in directorul current. Acestea se numesc: sample.class, add.class si sub.class . De ce s-a intamplat asta? Noi am compilat un singur program .java si acesta a creat 3 fisiere .class -> Motivul pentru care s-a intamplat asta se numeste Legarea dinamica. Astfel fisierele cu clasele sunt incarcate si executate numai in perioada de functionare si numai atunci cand exista o necesitate de anumite clase.

### **37. Considerati un exemplu de polimorfism si comportare polimorfica in Java**

```
class Shape2D { ... }
class Circle extends Shape2D { ...}
class Rectangle extends Shape2D { ... }
Shape2D s1, s2;
// atribuii polimorfice
// nu este necesar cast explicit
s1 = new Circle();
s2 = new Rectangle();
Circle c1;
//urmatoarea linie va genera eroare la compilare deoarece s1 pastreaza o referinta la un obiect de tip
//Circle, iar s1 este de tipul Shape2D care nu este un subtip a lui Circle
// verificarea tipului se face la compilare
c1 = s1;

// este necesar cast explicit
```

c1 = (Circle) s1; // ok, explicit cast

### 38. Principalele tehnici de reutilizare (performanta acestora)

Buclele asigura un mod eficient de repetare a unei parti de cod de cate ori este nevoie. Java are trei tipuri de bucle de repetare, acestea sunt while, for si do-while.

While – se foloseste pentru a repeta un bloc de declarații în timp ce o condiție este adevărată. Condiția trebuie să devină falsă undeva în bucla, altfel nu se va termina. Aceasta se foloseste cand nu se stie dinainte numarul de iteratii necesar, asa ca de fiecare data inainte de intrarea in bucla se verifica conditia si daca aceasta este adevarata bucla se executa.

Do while - Când doriți să testați la sfârșitul pentru a vedea dacă ceva ar trebui să se repete, do .. while este alegerea corecta. Este intotdeauna executat macar odata.

For – se foloseste cand se stie dinainte de cate iteratii este nevoie

Bucla de repetare are un efect considerabil in ceea ce priveste performanta programului. Sa ne uitam la cateva imbunatatiri a buclei de repetare:

- intotdeauna sa se foloseasca o data de tip intreg ca indexul variabilei din bucla oricand este posibil, deoarece este eficient cand se compara folosind „byte” sau date de tip „short”. Astfel, cand folosim „byte” sau date de tip „short” ca indexul variabilei din bucla, acestea implica implicit transferul tipului catre tipul „int”.

- cand se folosesc matrice, este eficient sa se copieze folosind System.arraycopy() in locul folosirii unei bucle de repetare.

- intotdeauna sa se evite orice poate fi realizat in afara buclei cum ar fi apelul metodelor, asigurarea valorilor unor variabile, sau testarea conditiilor

- apelurile metodelor sunt foarte costisitoare , astfel ca apelul acestora intr-un loop trebuie evitata

- este mai bine sa se evite accesarea elementelor matricilor intr-o bucla, cea mai buna varianta este sa se foloseasca variabile temporare in interiorul buclei si sa se modifice valorile matricii in afara buclei. Este mai rapid sa folosesti o variabila intr-o bucla decat sa accesezi un element dintr-o matrice

- incearca sa compari conditia de terminare cu zero daca folosesti non-JIT sau masina virtuala de tip HotSpot. Masinile virtuale JIT sau HotSpot sunt optimizate pentru bucle generale deci tu trebuie sa te mai deranjezi in ceea ce priveste conditia de terminare

- evita folosirea try-catch in interiorul buclelor, in schimb plaseaza bucla in interiorul try-catch pentru o performanta mai buna

### 39. Portabilitatea Java

Java este un limbaj de programare neutru architectural . Compilatorul Java genereaza cod de masina virtuala Java in loc sa genereze cod specific calculatorului care este folosit. Aceste coduri de masina virtuala, sau coduri de biti Java, sunt toate neutru arhitecturale. Codurile compilatorului

sunt executabile pe orice processor si system, asta doar daca interpretorul Java este instalat pe aceste sisteme. Deci, compilatorul genereaza instructiuni pe coduri de biti, care nu au nimic de lucru cu o arhitectura a unui calculator particular. Mai degraba, aceste coduri au fost create in asa fel incat sa fie usor de interpretat pe orice masina virtual si usor de translatat in cod negativ.

Librariile care sunt o parte din system defines interfete portabile.

Cea mai buna parte este ca sistemul Java este insusi portabil. Noul compilator este scris in Java si perioada de functionare este scrisa in C cu o limita de portabilitate bine realizata.

Astfel, fiind neutru arhitecturale, aplicatiile Java sunt ideale pentru un mediu cum este internetul. Portabilitatea softwareului peste toate platformele de calcul cu avantaje peste C/C++ este indiscutabila.

#### 40. Use case si diagrame use case

Use-cases (cazuri de folosire) – au fost introduse in SE de Ivar Jacobson (in 1992). Reprezinta un concept central in Relational Unified Process (RUP) si UML.

Reprezinta un set de scenarii referitoare la cum functioneaza sistemul. Ajuta la descoperirea: entitatilor sistemului, a actorilor de sistem, atributelor si a comportamentului.

UML a introdus diagramele use case pentru a captura grafic actorii de sistem, cazurile de utilizare si relatiile acestora.

Actorul cazului de utilizare – o functie, o persoana umana, o masina sau un soft care ruleaza in timpul functionarii sistemului de operare. Actorul intercatoneaza (comunica) cu sistemul.

Identificarea actorului cazului de utilizare – se identifica actorii principali si cum interactioneaza cu sistemul pentru a obtine functionalitate; se identifica si se clasifica alti actori.

Un caz de utilizare descrie modul in care actorul intercatoneaza cu sistemul. Cazurile de utilizare sunt construite dupa ce s-au identificat actorii primari.

Exemplu “verificare sold”:

Descriere caz de utilizare – clientul introduce cardul in ATM

-sistemul cere pinul

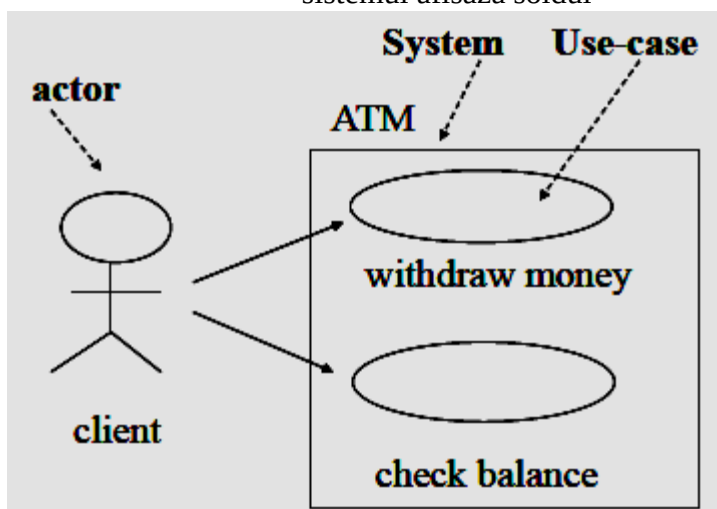
-sistemul valideaza pinul

-sistemul cere realizarea unei operatii. Clientul selecteaza “chverificare

sold”

-sistemul comunica cu retea ATM

-sistemul afisaza soldul



Tipuri de diagrame use case:

-interaction diagrams – descriu modul de colaborare al grupurilor de obiecte

-sequence diagrams – arata obiectele implicate in interactiune

-activity diagrams – descriu secvente de activitati

-package diagrams – ajuta la descompunerea sistemelor mari in subsisteme

-component diagrams – arata cum componentele sistemului relationeaza intre ele

-deployment diagrams – arata arhitectura fizica a sistemului

#### 41.String si String Buffer

Clasa String reprezinta caracterele de tip String. Stringurile sunt constant, valorile acestora nu pot fi modificate dupa ce au fost create.

String Buffer suporta stringuri variabile(schimbatoare) .

Deoarece obiectele string sunt neschimbatoare acestea pot fi partajate. De exemplu: String str ={"abc"} este echivalent cu char data[] = {'a', 'b', 'c'}; String str = new String(data);

Clasa String include metode de examinare individuala a caracterelor din secventa, pentru compararea stringurilor, pentru cautarea, pentru extragerea si pentru crearea unei copy a unui String cu toate caracterele transformate din litere mari in litere mici.

Limbajul Java ofera support special pentru operatorul de concatenare a stringurilor, ( + ) si pentru conversia altor obiecte in Stringuri. Concatenarea este implementata prin intermediul clasei StringBuffer, si metoda sa, "append".

Reprezentarea String a obiectelor:

- metoda toString() – returneaza reprezentarea String a unui obiect
- utila in testare si debugging
- rezultatul trebuie sa contina toate campurile obiectului

#### 42. -tehnici eager si lazy de instantiere,

- tehnica de instantiere eager
  - este o tehnica prin care instantele obiectelor sunt create inainte de a fi nevoie de ele
  - imbunatateste considerabil timpul de executie deoarece obiectele se afla in memorie asteptand sa fie folosite
- tehnica de instantiere Lazy
  - tehnica de conservare a memoriei
  - amana instantierea unor obiecte pana in momentul in care este nevoie de obiectele respective
  - in Java exista 2 categorii de instantiere de tip Lazy:
    - Lazy class loading
      - incarca clasele in memorie doar in momentul in care are loc o referinta la acestea pentru prima data. Acest lucru se poate intampla la primul apel al unei metode statice a clasei respective (ex. aClass.someStaticMethod()) sau la prima folosire a operatorului "new" pentru crearea unei instante a clasei (ex. AClass aClass = new AClass();)
      - utilizarea memoriei este redusa semnificativ; de exemplu daca o parte a programului nu este niciodata executata, clasele care sunt referite doar din acea parte a programului nu vor fi niciodata incarcate in memorie
    - Lazy object creation
      - are o legatura stransa cu lazy class loading
      - se refera la amanarea instantierii unei clase pana in momentul in care chiar este nevoie in cod de instanta respectiva

#### 43. -tratarea evenimentelor in Java,

- Toate actiunile utilizatorilor apartin unui set abstract de lucruri numite evenimente.
- Un eveniment descrie, suficient de detaliat, o anumită actiune a utilizatorului.
- Sistemul de executie Java notifică programul la aparitia unui eveniment de interes pentru acesta.
- Programele care gestionează interactiunea cu utilizatorul în acest fel sunt numite programe conduse de evenimente (event driven).
- Evenimentele din interfata cu utilizatorul includ apăsări de taste, deplasări ale mouse, clic-uri, s.a.m.d.

- Un program poate indica faptul că îl interesează numai anumite evenimente
- Ascultătorul (listener) de eveniment:
  - Este notificat la apariția unui eveniment
  - Apartine unei clase furnizate de către programatorul de aplicație
  - Metodele sale descriu acțiunile de efectuat la apariția unui eveniment
  - Un program indică ce anume evenimente are nevoie să primească prin instalarea obiectelor ascultătoare de evenimente
- Sursa unui eveniment:
  - Sursele de evenimente raportează asupra evenimentelor
  - La apariția unui eveniment, sursa evenimentului notifică totii ascultătorii de evenimente

#### 44. -tipuri de colectii Java

- Bag (denumit si Multiset)
  - cea mai generala forma de colectie
  - colectie neordonata
  - pot sa contina elemente duplicate
  - implementeaza interfata Java Collection
- Sets
  - nu contine elemente duplicate
  - implementeaza interfata Java Set
  - sunt de 2 tipuri
    - Unsorted Sets
      - colectie neordonata
    - Sorted Sets
      - implementeaza interfata SortedSet
- Liste
  - colectie ordonata de elemente
  - elementele sunt indexate (incepand cu 0)
  - sunt permise elemente duplicate
  - implementeaza interfata Java List
- Maps
  - cunoscute si ca: functii, dictionare, siruri asociative (functions,dictionaries,associative arrays)
  - colectie neordonata de asociere (cheie,valoare)
    - cheile trebuie sa fie unice
    - valoarea poate sa fie orice entitate
  - implementeaza interfata Java Map
  - poate sa fie de tipul Sorted Map (ordonata) -> implementeaza interfata Java SortedMap

#### 45. -this

din cursuri: (abureli)

- invoca un constructor al acestei clase cu care se potriveste semnatura data
  - trebuie sa fie prima comanda (statement) in constructor
- din documentatia java (mie mi se pare mai clar)
- in interiorul unei metode instantia sau a unui constructor, cuvântul cheie “this” este o referinta la obiectul curent – obiectul al carui metoda sau constructor a fost apelat
  - folosind “this” este posibila referinta catre orice membru al obiectului curent

#### 46. -threaduri.implementarea threadurilor in Java;

- Thread: un singur curs secvential al controlului, numit si proces usor (lightweight), lansat din procesul principal

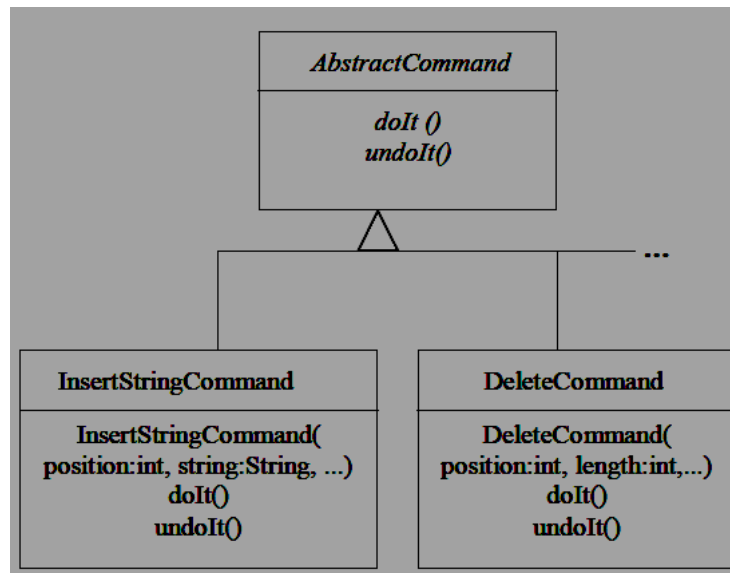
- Sunt procese paralele care rulează înăuntrul unui program
- În Java se pot crea thread-uri din program, așa cum se pot rula programe sub un sistem de operare
- Java creează thread-uri în două moduri:
  - Clasa extinde clasa **Thread** și îi suprascrie metoda **run()**.
    - La pornirea thread-ului, acesta invocă metoda **run()**, iar atunci când procesul revine din metoda **run()** thread-ul moare.
  - Clasa implementează interfața **Runnable**, care are o singură metodă: **run()**.
    - Clasa îi transmite o referință la sine însuși când creează un thread, apoi thread-ul apelează înapoi (calls back) metoda **run()** din clasă.
    - Thread-ul moare, ca la procedeul anterior, la revenirea procesului din **run()**.

#### 47. -tehnici de performanță în codificarea buclelor

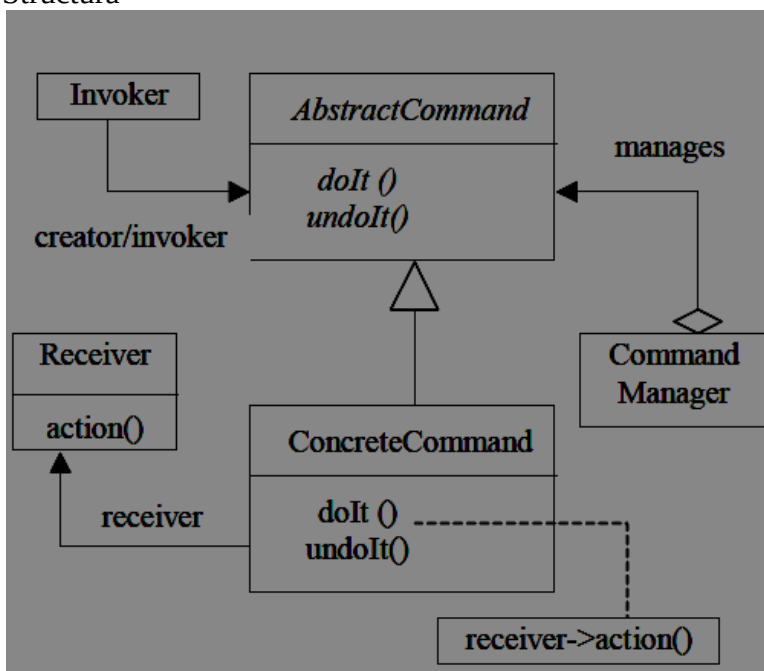
- utilizarea unei variabile de tip int ca și index al buclei
- pentru copierea unui array este întotdeauna mai eficientă folosirea funcției `System.arraycopy()` decât copierea array-ului folosind o buclă
- evitarea introducerii în buclă a oricăror lucruri ce pot fi făcute în afara acesteia, cum ar fi: apeluri de metode, asignarea de valori variabilelor sau testarea de condiții
- evitarea accesării directe a elementelor array-ului în buclă. Se recomandă folosirea unor variabile temporare în buclă, și modificarea elementelor array-ului să se facă în afara acesteia (este mai rapidă folosirea unei variabile în buclă decât accesarea unui element al array-ului).
- evitarea folosirii unui apel de metodă ca și condiție de terminare a buclei. Este preferabilă folosirea unei variabile temporare pentru verificarea condiției.
- când se folosesc în condiția de terminare a buclei mai multe expresii despartite prin operatorul “&&”, se recomandă punerea expresiei cu cea mai mare probabilitate de a fi falsă, în partea stângă, pentru a evita testarea următoarelor expresii în cazul în care prima este falsă.
- când se folosesc în condiția de terminare a buclei mai multe expresii despartite prin operatorul “||”, se recomandă punerea expresiei cu cea mai mare probabilitate de a fi adevărată, în partea stângă
- evitarea folosirii blocului try-catch în interiorul buclei; este mai performantă punerea buclei în interiorul unui bloc try-catch

#### 48. -command Design Pattern.

- de tip Behavioral
- folosește obiecte pentru a reprezenta operații (comenzi)
- comenzile pot fi tratate și manipulate ca și obiecte
- o secvență de comenzi poate fi reprezentată ca și cozi, stive sau alte structuri de date (permite folosirea comenzii “undo”, repetarea iteratiei etc.)
- concepție a unui program de procesare a cuvintelor (comenzile “do” și “undo”)
- fiecare comandă este reprezentată ca un obiect având metodele “do” și “undo”
- mod de lucru:
  - cere procesorului de cuvinte să execute ceva
  - creează un obiect, instanță a unei subclase a clasei `AbstractCommand`, care corespunde comenzii (trimite toată informația necesară constructorilor instanței)
  - apelează metoda `doIt()` pentru a executa comanda
  - pune comanda în structura de date pentru a reține istoria comenzilor

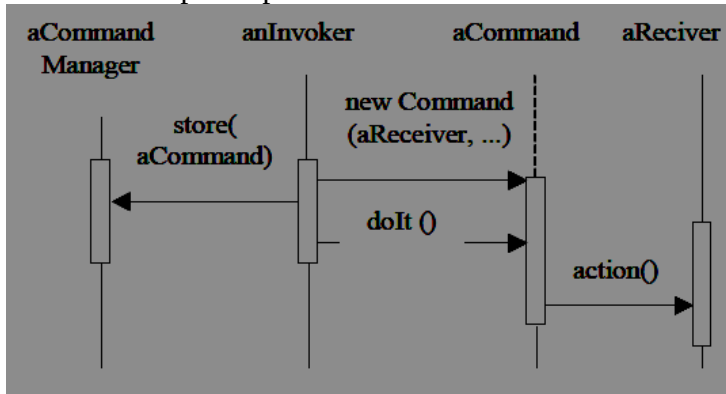


- Structura



- Participanti:
  - AbstractCommand
    - super-clasa claselor care incapsuleaza comenzile
    - defineste minimal o metoda *doIt* care este suprascrisa (specializata) in subclase
    - celelalte metode (undo) pot fi de asemenea definite de clasa AbstractCommand
  - ConcreteCommand
    - clasa concreta care incapsuleaza o comanda anume
    - defineste legatura dintre obiectul receiver si o actiune
    - implementeaza *doIt* prin invocarea operatiei/operatiilor corespunzatoare din clasa Receiver
    - celelalte clase invoca comanda printr-un apel al metodei *doIt* al clasei
    - constructorul obiectului, in mod normal, furnizeaza toti parametri necesari comenzii
  - Invoker

- o clasa avand acest rol, creaza obiecte de tipul ConcreteCommand daca este nevoie de invocarea unei comenzi
- poate apela metodele *doIt* a acelor obiecte sau poate lasa aceasta responsabilitate clasei CommandManager
- Receiver
  - stie cum sa faca operatia
  - de exemplu, poate fi un document, o aplicatie
- CommandManager
  - clasa responsabila pentru manipularea unei colectii de obiecte de tip Command create de obiectul/-ele de tip Invoker
  - poate fi independenta de aplicatie (reutilizabila)
- Colaborarea participantilor

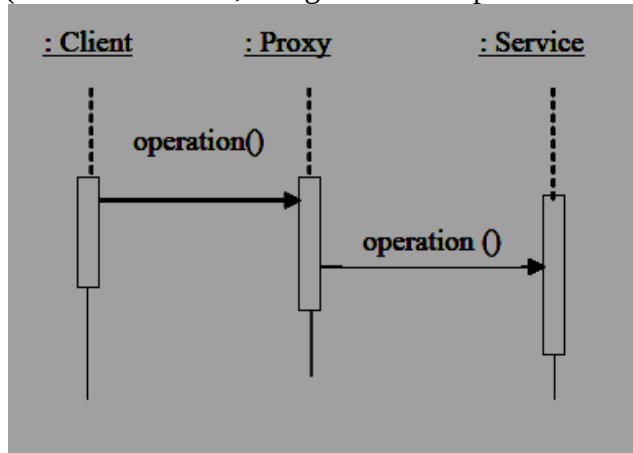


- Consecinte:
  - deconecteaza obiectul care invoca operatia de cel care stie sa o efectueze
    - invoker-ul si executorul sunt obiecte diferite
    - separarea permite flexibilitatea in sincronizarea si secventierea comenzilor (comenzile pot fi manipulate ca orice alte obiecte)
  - comenzile sunt obiecte ale unei clase primare (Commands are first-class objects )----
  - comenzile sunt asamblate in comenzi compuse
  - noile comenzi pot fi usor adaugate fara a le schimba pe cele existente

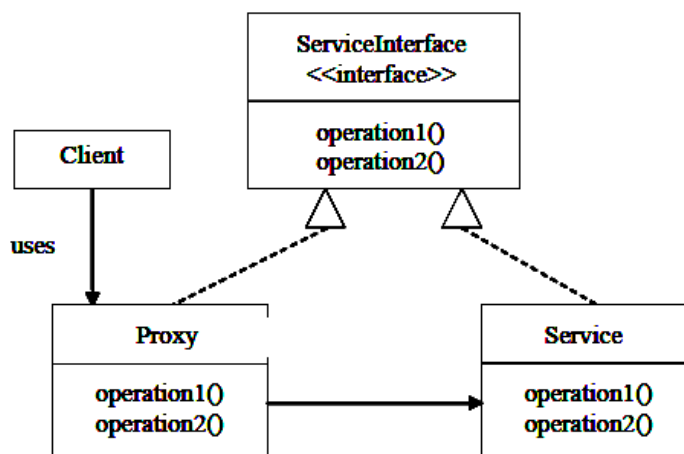
#### 49. -Proxy Design Pattern.

- de tip structural
- folosit pentru crearea de obiecte surogat care inlocuiesc alte obiecte (de obicei costisitor, fapt ce nu are nevoie de o instantiere imediata)
- clientii proxy nu stiu ca folosesc un surogat in loc de obiectele reale de care au nevoie
- obiectele proxy reprezinta obiecte reale
- obiectele proxy definesc aceleasi operatii ca cele reale (clasa Proxy si clasa ce o reprezinta – clasa Service - au aceeasi interfata sau aceeasi superclasa)

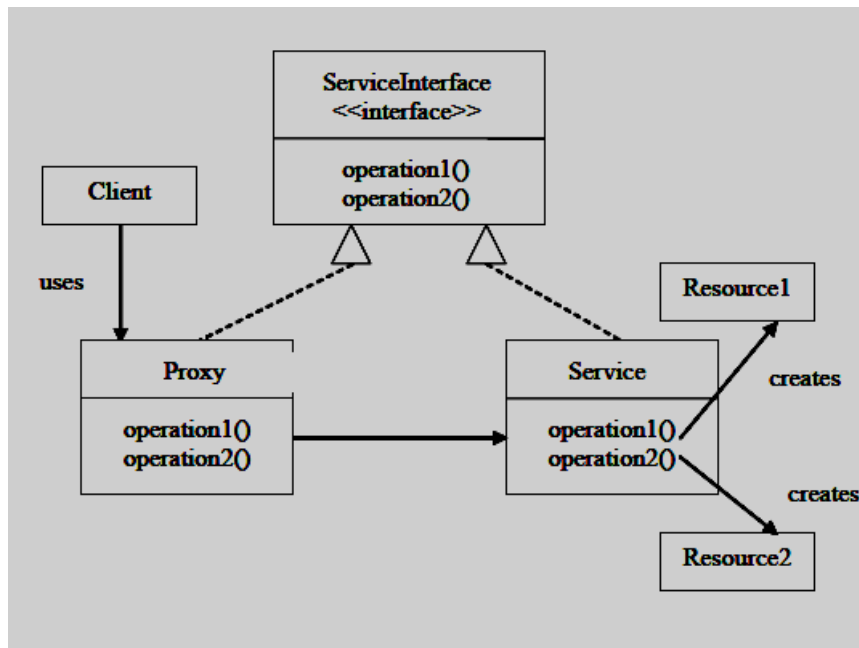
- operatiile Proxy implementeaza aspectul de gestionare (management aspect) al operatiei reale (daca este necesar, delega executia operatiei obiectului real)



- tipuri Proxy:
  - reprezentant al unei operatii costisitoare
    - timp lung de executie
    - proxy-ul returneaza un raspuns rapid, dar partial
  - proxy exterior (remote proxy)
    - reprezentant al unei operatii implementate pe un dispozitiv exterior (remote machine)
    - folosit in Java RMI, CORBA, RPC
  - proxy de acces
    - acceseaza controlul la resurse partajate
  - proxy virtual
    - reprezentant al unui obiect costisitor
    - obiectul costisitor este creat doar cand este necesar
    - amana instantierea obiectelor costisitoare pana in momentul in care este ceruta (instantiere lazy)
- diagrama de clase:



Nota: O clasa abstracta poate fi folosita in locul interfetei. In acest caz, clasele Proxy si Service sunt subclase ale clasei abstracte



- Participanti:
  - Service
    - furnizeaza serviciul complet
    - pentru Proxy Virtual, este responsabil sa-si creeze propriile resurse (Resource1, Resource2,...)
  - Client
    - foloseste serviciile furnizate de clasa Service
    - nu poate accesa direct operatiile si resursele Service, ci prin intermediul clasei Proxy
  - Proxy
    - o resursa indirecta dintre client si serviciu
    - amana instantierea obiectelor Service pana in momentul in care este nevoie de ele (Proxy Virtual)
    - Prin această indirectare, obiectul Client nu este conștient de faptul că obiectul Service nu a putut fi creat atunci când operațiunea este invocată și procesata de obiectul Proxy (Proxy Virtual)
    - obiectul Proxy este responsabil de crearea obiectului Service asociat de indata ce operatia a fost invocata (Proxy Virtual)
    - in Java, pattern-ul Proxy previne incarcarea clasei Service. El foloseste o referinta dinamica pt a accesa serviciul cand este necesar (Virtual Proxy)
      - referinta dinamica
        - incarcarea clasei are loc prin invocarea clasei utilizand string-ul cu numele acesteia
        - `Class.forName("ClassName")`
        - compilatorul nu este constient de aceasta si nu genereaza ca rezultat incarcarea clasei
    - Interfata Service
      - declara toate metodele implementate de Service si Proxy
- Consecinte
  - clientii nu sunt constienti de aranjamentele Proxy-ului
  - clasele ce sunt accesate prin proxy sunt incarcate in mod dinamic
    - doar cand este necesar (proxy virtual)
    - instantele acestora sunt create doar cand este nevoie de ele

## 50. -Good and bad casting usage.

Folosirea corecta a cast-ului: cand referinta unui obiect este cast-uita la:

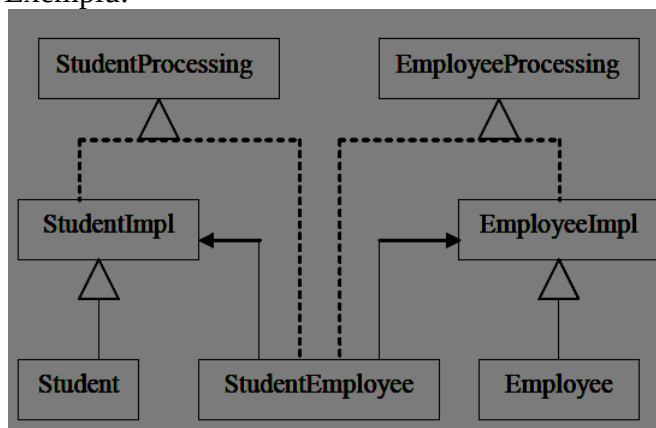
- referinta unei variabile a carei tip este acelasi cu tipul clasei de unde a fost instantiat obiectul
- referinta unei variabile a carei tip este o superclasa a clasei de unde obiectul a fost instantiat
- referinta unei variabile a carei tip este o interfata care este implementata de catre clasa de unde obiectul a fost instantiat
- referinta unei variabile a carei tip este o interfata care este implementata de catre superclasa clasei de unde obiectul a fost instantiat

Folosirea incorecta a cast-ului:

- cand se incearca cast-ul intre 2 obiecte care sunt total nerelationate (nu este relatie subclasa-superclasa sau clasa-interfata)
- la executie se va arunca o exceptie de tipul ClassCastException daca obiectul asupra caruia se face cast-ul nu este compatibil cu noul tip la care este cast-uit

## 51. -Describe what is delegation. Example. Where it is used?

- Delegația este un concept ce are legatura cu agregarea
- Un obiect complex format dintr-un set de obiecte de complexitate mai mica
- O metodă care este invocata în asociere cu partile delegate ale obiectului complex.....  
(A method that is invoked in association with the complex object delegates parts of its functionality to lower level component objects)
- folosita la mostenirea de tip simplu sau multiplu
- Exemplu:



```
public interface StudentProcessing {
 public double getAvgMarks();
 public double calculateAvgMarks();
}

public interface EmployeeProcessing {
 public double getSalary();
 public double calculateSalary();
}

public class StudentImpl implements StudentProcessing {
 protected double avgMarks;
 public double calculateAvgMarks() {...}
 public double getAvgMarks() {...}
}

public class EmployeeImpl implements EmployeeProcessing {
 protected double salary;
 public double calculateSalary() {...}
}
```

```

public double getSalary() { ... }
}
public class Student extends StudentImpl {
// ... methods getAvgMarks() and calculateAvgMarks() are inherites
// ... avgMarks field is inherited
}
public class Employee extends EmployeeImpl {
// ... methods getSalary() and calculateSalary() are inherites
// ... salary field is inherited
}

```

```

public class StudentEmployee implements StudentProcessing, EmployeeProcessing {
protected StudentImpl studentImpl;
protected EmployeeImpl employeeImpl;
public StudentEmployee() {
studentImpl = new StudentImpl();
employeeImpl = new EmployeeImpl();
// ...other constructor statements
}
public double getAvgMarks() {
// delegation is used
return studentImpl.getAvgMarks();
}
public double getSalary() {
// delegation is used
return employeeImpl.getSalary();
}
// ... same for calculateAvgMarks() and calculateSalary()
}

```

- metodele care folosesc delegatia:
  - getAvgMarks() si getSalary()
  - calculateAvgMarks() si calculateSalary()
  - fiecare metoda isi delega sarcina altui obiect

## 52.Downcasting techniques

Downcasting=casting-ul unei variabile referinta la un subtip al tipului sau declarat

-castul explicit in java este permis la momentul compilarii

-validitatea castului explicit este verificata la runtime. In caz de invaliditate va genera exceptie ClassCastException

### Tehnici de downcasting:

- **abordare pesimista (Pessimistic approach)**

```

if(s2 instanceof Circle)
{
 Circle cref = (Circle) s2;
}

```

```

 }
else {
// ... do something else

```

- **abordare optimista (Optimistic approach)**

```

try {
// ...
Circle cref = (Circle) s2;
// ...
} catch (ClassCastException e) {
// ... do something
}

```

-exemplu:

Presupunem ca avem clasele:

```
class Shape2D { ... }
```

```
class Circle extends Shape2D { ... }
```

```
class Rectangle extends Shape2D { ... }
```

clasa Rectangle defineste metoda `getNoCorners()`, care nu este definita in Shape2D

```

Shape2D s1 = new Rectangle();
// ...
// compile error
//validitatea metodelor invocate este verificata statistic
int nc = s1.getNoCorners();
// ...

```

```

Shape2D s1 = new Rectangle();
// ...
if(s1 instanceof Rectangle) {
Rectangle r1 = (Rectangle) s1;
int nc = r1.getNoCorners();
// ...
}

```